기기 4대 분담 실행 계획

AutoGluon 변형, 단일 모델, 튜닝, 딥러닝, 앙상블을 기기 4대에 나눠 약 10시간에 끝내는 계획

핵심. 모든 기기가 같은 저장소 코드(`Yu-Jusang/galaxy-sfr-automl`)로 같은 분할·같은 5-fold·같은 변수를 계산한다. 각 기기는 명령 한 줄(`python scripts/jobs.py run <기기>`)로 맡은 작업을 돌리고, 결과 zip을 A에 보낸다. A가 합쳐서 앙상블과 최종 선택을 한다.

1 왜 이렇게 나누나

1. **변수 효과는 튜닝 없는 모델로 판정할 수 없다.** 원래 최종 모델 설정(튜닝 없는 LightGBM)에 문헌 묶음이나 친구 변수를 하나씩 넣으면 모두 CV가 0.1–0.2 나빠졌다(묶인 동반 은하 L7만 거의 같음). 하이퍼파라미터가 25개 변수에 맞춰져 있어서다. 그래서 새 변수는 **튜닝**(변수 묶음 포함 여부도 Optuna가 고름)과 **AutoGluon**(설정이 변수에 맞춰 바뀜)으로 판정한다.
2. **AutoGluon은 두 축으로 돌린다.** 변수 축(어떤 변수를 넣나)과 전처리 축(목표 변환, 손실, 가중치, 프리셋)이다. 축마다 한 가지씩만 바꿔야 효과를 따로 읽을 수 있다.
3. **기기 성격에 맞춘다.** 딥러닝·사전학습은 GPU(Mac MPS 또는 CUDA), 튜닝은 CPU 코어 수, AutoGluon은 RAM과 코어 수가 중요하다.

2 변수 세트

세트	개수	구성
F0	13	CSV 원래 입력 열
F3	22	F0 + 물리 파생 9
F5	25	원래 최종 모델 변수
F5L	62	F5 + 문헌 묶음 L1–L7
F5S	52	F5 + 친구 변수 27
F5LS	89	F5 + 문헌 + 친구
F5X	153	만든 변수 전부(F5 + L + S + A1 + A2 + G + N + MSSM)

정확한 수식은 `report/feature_definitions.pdf`.

3 기기별 작업

A — 조정용 노트북 (Windows, 16스레드, RAM 16 GB, GPU 없음) · 약 10.5시간

작업	내용	시간
J01	F5LS 단일 모델 7개: 규제 LightGBM, XGBoost, CatBoost, ExtraTrees, RF, MLP, 로그 목표 LightGBM	1.0 h
J02	선형·커널 나머지 3개: Nystroem 커널 Ridge, KNN 두 가지	0.5 h
J03–J06	AutoGluon 변수 축: F5L, F5S, F3, F0	6.0 h
J18	AutoGluon 전처리 축: 제곱근 목표, 스택 2층 (F5LS)	3.0 h
마지막	결과 합치기, 앙상블, 최종 선택, 검증셋 1회 평가, 보고서·GitHub	2 h

B — Apple Silicon Mac (MPS) · 약 8시간

작업	내용	시간
J07	딥러닝·커널·사전학습 6종 × 변수 2세트(F5, F5LS) = 12개: Tweedie MLP, TabM, RealMLP(원 목표·로그 목표), xRFM, TabPFN v2	4.0 h
J08	AutoGluon F5LS, 4시간 예산(모델 묶음을 더 많이 학습)	4.0 h

C — CPU 기기 (RAM 16 GB 이상) · 약 9시간

작업	내용	시간
J09	LightGBM nested Optuna 튜닝 50회/fold	2.5 h
J10	XGBoost nested Optuna 튜닝 40회/fold	3.0 h
J11	CatBoost nested Optuna 튜닝 25회/fold	3.5 h

튜닝 대상: Tweedie p, 이웃 반경·k·중심 판정 반경, 과밀도 문턱, 거대 은하 가중치, 모델 하이퍼파라미터, 그리고 **문헌 묶음 L1–L7과 친구 변수 S 각각의 포함 여부**. 바깥 5-fold의 학습부 안에서만 3-fold로 탐색하므로 평가 fold를 보지 않는다.

D — CUDA GPU 기기 (Colab Pro, 연구실 서버 등) · 약 9.5시간

작업	내용	시간
J12–J13	AutoGluon 변수 축: F5LS, F5X(전부)	3.0 h
J14–J16	AutoGluon 전처리 축(F5LS): Tweedie 모델 추가, 로그 목표, 거대 은하 가중치 2배	4.5 h
J17	AutoGluon `extreme_quality`(TabPFNv2, TabM, RealMLP, Mitra 포함) — CUDA GPU 필요	2.0 h

GPU가 없으면 J17은 실패할 수 있다. 그때는 J12–J16만 돌린다.

4 AutoGluon 실행 목록 전체

이름	변수	축	바꾼 것
AG1_F0	F0	변수	원래 13열
AG2_F3	F3	변수	+ 물리 파생
AG3_F5	F5	기준	**완료: CV 7.317**
AG8_F5L	F5L	변수	+ 문헌 묶음
AG9_F5S	F5S	변수	+ 친구 변수
AG10_F5LS	F5LS	변수	+ 문헌 + 친구
AG11_F5X	F5X	변수	만든 변수 전부
AG12_F5LS_tweedie	F5LS	손실	Tweedie LightGBM · XGBoost 추가
AG13_F5LS_log	F5LS	목표	log1p + smearing
AG14_F5LS_sqrt	F5LS	목표	제곱근 변환
AG15_F5LS_wmassive	F5LS	가중치	$\log M_* > 11.5$ 은하 2배
AG16_F5LS_stack2	F5LS	스태킹	2층 추가
AG17_F5LS_long	F5LS	시간	4시간
AG18_F5LS_extreme	F5LS	프리셋	extreme_quality (GPU)

5 공통 규칙

- 모든 기기: 같은 고정 80/20 분할, 같은 11층 층화 5-fold(seed 42). AutoGluon도 bagging fold를 이 5-fold로 강제한다.
- 선택은 CV로만 한다. 검증셋은 A가 마지막에 한 번만 평가한다.
- 결과 zip의 OOF 파일에는 학습 은하의 정답 SFR이 들어 있으므로 공개된 곳에 올리지 않는다.
- 코드는 고치지 않는다. 오류가 나면 로그를 그대로 공유한다.
- 최종 선택 규칙(실험 전에 고정): 모든 단일 모델 · 튜닝 · AutoGluon · 앙상블 가운데 **CV RMSE가 가장 낮은 것**. 앙상블 가중치도 CV 안에서 학습한다.

6 실행 순서

1. **모든 기기:** 저장소 clone, 환경 설치, CSV를 `data/`에 둔다(HELPERS.md).
2. **각 기기:** `python scripts/jobs.py run <기기>`. 꺼지면 같은 명령으로 이어서 실행한다.
3. **각 기기:** `python scripts/jobs.py pack <기기>` → `results/pack_<기기>.zip` 을 A에 보낸다.
4. **A:** `python scripts/jobs.py unpack pack_B.zip pack_C.zip pack_D.zip` → `python scripts/a04_ensemble.py`.
5. **A:** 보고서 PDF, 코드 zip, GitHub 페이지를 갱신한다.